Noosphere Backend — Third-Party Integrations Document

Noosphere Backend — Third-Party Integrations Document

> Generated from a full static scan of the repository (dependencies in `package.json`, `src/config/*`, `src/utilities/*`, and every feature module that calls out to an external service). This document describes integrations **as implemented in code today**, including ones that are configured but not actually wired up, and ones that appear to be legacy/dead code.

1. Integration Inventory (at a glance)

#	Provider / Service	Purpose	Status	Primary files
1	**Stripe**	Invoice payment processing (PaymentIntents + webhooks)	✅ Active, most mature integration	`src/features/billing/application/stripeBillingService.js`, `billingController.js`, `billingRoute.js`
2	**AWS S3**	File/document storage (uploads, downloads, presigned URLs)	✅ Active	`src/utilities/s3.js`
3	**AWS SES v2**	Transactional email (tenant-branded "from" addresses)	✅ Active	`src/utilities/ses.js`, `src/utilities/notificationProcessor.js`
4	**SMTP via Gmail (Nodemailer)**	Transactional email (invoices, payment links)	✅ Active, runs **in parallel** with SES	`src/utilities/nodemailer.js`
5	**AWS CloudWatch**	Infrastructure metrics for the in-app	✅ Active	

#	Provider / Service	Purpose	Status	Primary files
		Performance dashboard (EC2 + RDS)		`src/utilities/cloudWatch.js`, `src/features/performance/*`
6	**Google OAuth 2.0** (`passport-google-oauth20`)	"Sign in with Google"	⚠️ Configured but **not wired into the app** (dead code)	`src/config/passport.js`
7	**Socket.IO**	Realtime messaging & notifications	✅ Active (self-hosted, not a third-party *service*, but a third-party *library* — included for completeness)	`src/config/socket.js`
8	**RabbitMQ** (`amqplib`)	Message queue	❌ Fully commented out / never enabled	`src/config/rabbitMQ.js`
9	**Swagger / OpenAPI** (`swagger-jsdoc`, `swagger-ui-express`)	API documentation	✅ Active (dev tooling, not a runtime integration)	`src/config/swagger.js`
10	**Speakeasy + QRCode**	TOTP-based 2FA (secret generation + QR provisioning)	⚠️ Dependencies present; usage not confirmed in this scan (see §8)	`package.json` only

---

2. Stripe (Payments)

Purpose: charge tenants for invoices via a hosted payment-link flow, using Stripe PaymentIntents and Stripe.js on the frontend.

SDK: `stripe` npm package (v22). Client is lazily instantiated per-call (not a singleton) via a getter:

```
get stripe() {
    if (!process.env.STRIPE_SECRET_KEY) {
        throw new Error("STRIPE_SECRET_KEY is not configured");
    }
```

```
    return new Stripe(process.env.STRIPE_SECRET_KEY);
}
```

This means a missing `STRIPE_SECRET_KEY` fails fast with a clear error at the point of use rather than at process startup.

2.1 Credentials / configuration

Env var	Purpose
`STRIPE_SECRET_KEY`	Server-side API key used to construct the Stripe client
`STRIPE_WEBHOOK_SECRET`	Used to verify inbound webhook signatures (`stripe-signature` header)

Currency is hardcoded to `usd` (`const CURRENCY = "usd"` in `stripeBillingService.js`) — there is no multi-currency support anywhere in the Stripe integration.

2.2 Customer management

`ensureCustomer(tenant)` lazily creates a Stripe Customer for a tenant on first use and persists `Tenant.stripeCustomerId`. It guards against a race condition where two concurrent requests might both see `stripeCustomerId === null`:

- Creates the Stripe customer optimistically
- Persists it with a conditional `updateMany({ where: { id, stripeCustomerId: null } })` (only succeeds if nobody beat it to the update)
- If the conditional update reports 0 rows affected, re-fetches the tenant's now-current `stripeCustomerId`, deletes the just-created duplicate Stripe customer (`stripe.customers.del(...).catch(() => {})`, best-effort), and returns the winning customer ID instead

2.3 Payment flow (PaymentIntents API — modern integration, not Stripe Checkout)

Public-facing endpoints under `/api/v1/billing/stripe/*`, gated only by payment-link tokens (not JWT auth) and throttled via `paymentLinkRateLimiter` (see ARCHITECTURE.md §5.3):

1. **`POST /stripe/create-payment-intent`** (`{ token }`) — resolves the invoice for the token via `invoiceService.validatePaymentToken`, computes `expectedAmount(invoice) = round(invoice.total * 100)` (dollars → cents), and either:
   - reuses an existing `PaymentIntent` on the invoice if it's still in a reusable state (`requires_payment_method`, `requires_confirmation`, `requires_action`, `processing`), or
   - creates a new one, attaching `metadata.invoiceId` for later verification, and persists `Invoice.stripePaymentIntentId`

    - Returns `{ clientSecret, paymentIntentId, amount, currency }` for the frontend to complete with Stripe.js — **the server never sees raw card data** (standard Stripe Elements/PaymentIntents pattern).
2. **`POST /stripe/confirm-payment`** (`{ token, paymentIntentId }`) — a client-triggered confirmation path (defense in depth alongside the webhook, in case the webhook is delayed or the client wants immediate feedback):
    - Retrieves the PaymentIntent from Stripe (`expand: ["latest_charge"]`)
    - `assertPaymentIntent()` validates: `paymentIntent.metadata.invoiceId` matches the invoice, `status === "succeeded"`, and `amount_received`/`currency` match the invoice exactly (defends against tampering/mismatched amounts)
    - `activate()` runs inside a **Serializable-isolation Prisma transaction with retry-on-conflict** (handles `P2034` write-conflict errors, up to 3 attempts with backoff) — this specifically guards against the webhook and this endpoint both trying to settle the same invoice concurrently
    - Marks the invoice `Paid`, records a `Payment` row (`gateway: "stripe"`, `transactionId` = PaymentIntent ID, `transactionRef` = charge ID), upserts a `PaymentMethod` from the card details on the charge, and marks the invoice's payment-link token used
3. **`POST /stripe/webhook`** — the authoritative, asynchronous settlement path:
    - Route-level `express.raw({ type: "application/json" })` middleware preserves the exact raw body bytes required for Stripe signature verification (this is why the global JSON body-parser in `server.js` explicitly excludes this path)
    - Rate-limited via `webhookRateLimiter` (120 req/min)
    - Supports **both** Stripe's legacy (v1) webhook events (verified via `stripe.webhooks.constructEvent`) **and** the newer "thin" v2 event notifications (verified via `stripe.parseEventNotification`, then `event.fetchEvent()` to hydrate the full payload) — the code checks `payload.object === "v2.core.event"` to decide which path to take
    - For thin v2 events, non-payment-related event types are acknowledged and discarded immediately (`/^v1\.(payment_intent|charge)\./` filter) without fetching the full event, to avoid unnecessary API calls
    - **Idempotency**: every verified event is first inserted into `StripeWebhookEvent` keyed by Stripe's `eventId`; a Postgres unique-constraint violation (`P2002`) on a replayed event causes the handler to return early without reprocessing — this is the sole idempotency guard against Stripe's at-least-once webhook delivery
    - Event types handled: `payment_intent.payment_failed` (records a failed `Payment`), `payment_intent.succeeded` (settles the invoice — same activation path as `/confirm-payment`, so whichever of the two fires first "wins" and the other is a no-op thanks to the `status === "Paid"` early-return and the serializable transaction), `charge.refunded`, `charge.dispute.created`/`charge.dispute.closed`. All other event types are acknowledged (200) and ignored.
    - Always returns HTTP 200 on successful verification+processing so Stripe doesn't retry; returns 400 only on signature/parsing failure (which *should* trigger a Stripe retry, since it may indicate a transient issue — though a bad signature will also retry uselessly).

2.4 Legacy path

`POST /api/v1/billing/pay-payment-link` is kept only as a **410 Gone** stub ("Deprecated; cannot activate a payment… Use the Stripe PaymentIntent endpoints instead") — a manual/non-Stripe payment recording flow that has been fully retired but the route is left mounted for clients still hitting the old URL, to return a clear deprecation message instead of a 404.

2.5 Data model touchpoints

`Tenant.stripeCustomerId`, `Invoice.stripePaymentIntentId` (unique), `StripeWebhookEvent` (idempotency ledger), `Payment` (settlement records with `gateway: "stripe"`), `PaymentMethod` (tokenized card metadata — `gatewayToken`, `cardType`, `lastFourDigits`; **no raw PAN/CVV is ever persisted**, consistent with PCI-DSS SAQ-A scope for a PaymentIntents/Elements integration).

2.6 Risks / observations

- `STRIPE_WEBHOOK_SECRET` absence is checked defensively in the controller (returns 500 with a clear message) — good; but there's no equivalent startup-time check, so this failure mode is only discovered the first time a webhook fires in a misconfigured environment.
- The Stripe client is re-instantiated on every access (`get stripe()` creates a `new Stripe(...)` each time) rather than cached as a singleton — functionally correct but slightly wasteful; worth memoizing if this becomes a hot path.
- Amount handling: `Invoice.total` is a plain `Int` (assumed already in major currency units, e.g. dollars) and multiplied by 100 to get Stripe's expected minor-unit amount — this convention is easy to get wrong if any other code path assumes `Invoice.total` is already in cents (see DATA_MODEL.md §7.9 for the broader currency-modeling inconsistency across the schema).

---

3. AWS S3 (File Storage)

Purpose: store uploaded documents (client documents, staff documents, clinical report exports, pipeline submissions) and serve them back via presigned URLs.

SDKs: `@aws-sdk/client-s3`, `@aws-sdk/s3-request-presigner`, plus `multer` + `multer-s3` for handling multipart uploads directly to S3 without buffering to local disk.

3.1 Configuration

Env var	Purpose
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`	Static credentials (not an IAM role/ instance-profile pattern)
`AWS_REGION`	S3 client region
`AWS_ENDPOINT`	Custom endpoint override (referenced in code; not present in the local `.env`

Env var	Purpose
	sample — likely used for local/dev S3-compatible storage such as MinIO/ LocalStack, or left blank for real AWS)
`S3_BUCKET_NAME`	Target bucket for all uploads

The client is constructed with `forcePathStyle: true`, which is required for S3-compatible endpoints (e.g. MinIO) and also works fine against real AWS S3.

3.2 `S3Service` capabilities (`src/utilities/s3.js`)

- **`getUploadMiddleware()`** — returns a configured `multer` instance wired to `multerS3` storage:
  - Object keys: `${Date.now()}-${file.originalname}` (timestamp-prefixed, not content-hashed or UUID-based — collision risk is low but non-zero under high concurrency with identical filenames within the same millisecond, and the original filename is preserved verbatim in the key, which can leak PII/internal naming or contain special characters)
  - `contentType: multerS3.AUTO_CONTENT_TYPE` — S3 stores whatever MIME type `multer`/the browser reports for the upload, not a server-validated value
  - `fileFilter`: allow-list combining MIME type (`image/jpeg`, `image/png`, `image/gif`, `image/webp`, `application/pdf`, `video/mp4`, `video/mpeg`) **or** file extension (`.doc`, `.docx`, `.xls`, `.xlsx`, `.ppt`, `.pptx`, `.pdf`, `.csv`, plus the image/video ones) — a file passes if *either* check succeeds, which is permissive (e.g. a `.docx` with a spoofed/incorrect MIME type still passes on extension alone, and vice versa)
  - Size limit: 50MB
- **`uploadBuffer(key, buffer, contentType)`** — server-side upload of an in-memory buffer (used for generated invoice PDFs — see `invoicePdfGenerator.js`), `ACL: "private"`, defaults to `application/pdf`
- **`getPresignedUrl(key, expiresIn = 3600)`** — time-limited signed GET URL (1 hour default) for private object access without exposing AWS credentials to the client
- **`getObjectStream(key)`** — server-side streamed read (e.g. for proxying a download through the API)
- **`deleteObject(key)`** — hard delete from the bucket

3.3 Known gap (cross-referenced from ARCHITECTURE.md §12.4)

The MIME `type` persisted on document metadata (e.g. `ClientDocuments.documentDetails.type`) is whatever the **frontend** sends in the JSON request body describing the upload — `S3Service` does not independently derive or validate the MIME type server-side from the actual uploaded file/extension. This was the root cause investigated in a prior session (a `.docx` file incorrectly tagged `image/jpeg`). The S3 upload `fileFilter` validates that *a* file was allowed through, but nothing reconciles that with the separately-submitted `documentDetails.type` metadata used for rendering elsewhere in the product.

4. Email — Two Parallel Providers

The codebase sends transactional email through **two independent, uncoordinated systems**. This is a notable architectural inconsistency worth reconciling.

4.1 AWS SES v2 (`src/utilities/ses.js`, `EmailService` / `emailService` singleton)

- SDK: `@aws-sdk/client-sesv2` (`SESv2Client`, `SendEmailCommand`)
- Credentials: `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_REGION` (same AWS credentials as S3 — no separate SES-scoped key)
- **Tenant-branded sending**: `sendTenantEmail({ tenantSlug, to, subject, html, text, replyTo })` synthesizes a from-address as `${tenantSlug}@noospherehub.net` with a display name derived by title-casing the slug (`formatTenantName`) — this requires the `noospherehub.net` domain (and presumably a wildcard/catch-all or per-tenant verified identity) to be SES-verified for sending.
- **Attachment support**: `sendTenantEmailWithAttachment(...)` hand-builds a raw MIME multipart/mixed message (`buildRawEmail`) — base64-encodes the attachment, splits it into 76-character lines per MIME convention, and sends it via SES's `Content.Raw` mode (required because SES's `Content.Simple` mode doesn't support attachments). Enforces a 10MB attachment limit (SES's raw-message limit) and requires a `Buffer` input.
- **Header injection defense**: `sanitizeHeader()` strips `\r`/`\n` from any value interpolated into a raw email header (from/subject/reply-to/attachment filename) — mitigates classic SMTP/MIME header injection via user-controlled input (e.g. a tenant slug or subject line containing embedded newlines).
- **Email validation**: `validateEmail`/`validateEmails` — basic regex format check, throws on the first invalid recipient.
- **Retry helper**: `sendWithRetry(sendFunction, maxRetries=3, initialDelay=1000)` — exponential backoff, but explicitly does **not** retry on 4xx-coded errors (client/validation errors are typically not transient). This helper exists on the class but its actual call sites were not confirmed to be exercised by every send path in this scan — worth verifying `sendTenantEmail` callers wrap calls in it where retry semantics matter.
- **Primary caller**: `src/utilities/notificationProcessor.js` uses `emailService.sendTenantEmail` for the `email` channel of in-app notification types.

4.2 SMTP via Gmail (`src/utilities/nodemailer.js`, `MailService` static class)

- SDK: `nodemailer`, hardcoded to `smtp.gmail.com:465` (secure/TLS), authenticated with `MAIL_USER`/`MAIL_PASS` (a Gmail account, likely using an app-specific password given 2FA requirements for SMTP auth on Gmail)
- Single `sendMail(to, subject, text, html, attachments)` static method, always sends `from: "Noosphere" <MAIL_USER>` — **not tenant-branded**, unlike the SES path
- Success is verified by checking `info.accepted.includes(to)`; failures are swallowed into a `{ success: false, error }` return value rather than thrown — **callers must remember to check `result.success`** (confirmed

pattern in `invoiceService.js`/`invoiceManagementJob.js`, which do check this)
- **Primary callers**: `invoiceService.js` (payment-link emails), `invoiceManagementJob.js` (upcoming/due/overdue invoice reminders) — i.e., **all invoicing-related email currently goes through Gmail SMTP, not SES**, despite SES being the more "production-grade" AWS-native choice already available in the codebase.

4.3 Templates

`src/utilities/templateRenderer.js` renders HTML files from `src/templates/emails/` with simple variable substitution — used by the Nodemailer/invoice email paths. The SES/notification-processor path instead uses a small hardcoded `switch` statement in `notificationProcessor.js` (`buildTemplate()`, cases: `welcomeTenant`, `paymentSuccess`, default) that returns bare `<h1>...</h1>` strings — a much less developed templating approach than the file-based renderer, suggesting the notification-processor email path is either legacy or an unfinished migration (also flagged in ARCHITECTURE.md §10.2).

4.4 Risk summary

- **Two providers, two branding conventions, two failure-handling conventions** (thrown vs. returned errors) — a maintenance hazard; a bug fix or deliverability improvement (e.g. DKIM/SPF, bounce handling, suppression lists) applied to one path won't automatically apply to the other.
- Gmail SMTP for transactional invoice email at any scale risks Gmail's sending limits/reputation-based throttling — SES is generally the more appropriate channel for high-volume transactional mail and is already integrated; consolidating onto it (or at least routing invoicing through it) would likely improve deliverability.

---

5. AWS CloudWatch (Infrastructure Monitoring)

Purpose: power the in-app "Performance" dashboard (`src/features/performance/*`) with live EC2/RDS metrics, rather than requiring operators to leave the product to check the AWS console.

SDK: `@aws-sdk/client-cloudwatch` (`CloudWatchClient`, `GetMetricDataCommand`).

5.1 Configuration

Env var	Purpose
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_REGION`	Same AWS credentials pattern as S3/SES (no dedicated read-only monitoring credentials)
`EC2_INSTANCE_ID`	Which EC2 instance's metrics to query
`RDS_INSTANCE_ID` / `MY_DB_INSTANCE`	

Env var	Purpose
	Which RDS instance's metrics to query (two separate env vars appear to reference the DB instance — possibly redundant/legacy naming, worth confirming only one is actually read)
`RDS_TOTAL_MEMORY_GB` (default 16) / `RDS_TOTAL_STORAGE_GB` (default 100)	Used to convert CloudWatch's *free* memory/storage metrics into *used %* for dashboard gauges, since CloudWatch doesn't report total capacity directly
`CLOUDWATCH_USERNAME`	Present in `.env` but not referenced in `cloudWatch.js` in this scan — likely used elsewhere (e.g. IAM user display name in an ops runbook) or vestigial

5.2 `CloudWatchUtil` capabilities

- **Generic fetchers**: `getMetricData()` (single metric/dimension/period), `getMetricByAllPeriods()` (fans out the same metric across year/month/week/day windows in parallel via `Promise.all`, each with an appropriately coarser `period` — 1 day/1 hour/1 hour/5 min respectively), `getMultipleMetrics()` (batch fetch across arbitrary metric queries for a shared time range)
- **Downsampling**: `formatForChart()` buckets and averages data points down to a `maxPoints` cap (default 50) for smooth chart rendering regardless of the query's native resolution
- **EC2 metric shortcuts**: CPU, disk read/write bytes & ops, network in/out & packets, status-check-failed (instance/system/overall)
- **RDS metric shortcuts**: CPU, connections, free storage, freeable memory, read/write IOPS & latency & throughput, replica lag, swap usage, disk queue depth, network throughput
- **Dashboard-specific aggregations** purpose-built for known frontend widgets: `getGeneralMetrics`/`getGeneralTimeseries` (system speed from `NetworkOut`, latency from RDS `ReadLatency`/`WriteLatency`, uptime derived from the *percentage of hourly `StatusCheckFailed` checks that were zero* over the last 24h), `getApiErrorRate` (derives a monthly error/success percentage from `StatusCheckFailed` sums, assuming ~720 hourly checks/month), `getResourceMetrics`/`getResourceTimeseries` (CPU % directly, memory/storage % computed as `(total - free) / total`)

5.3 Risks / observations

- **"API error rate" and "uptime" are approximated from EC2 health-check failures**, not from actual HTTP 5xx response rates that the app already tracks natively in the `ServerRequest` table (see ARCHITECTURE.md §10.6). This means the Performance dashboard's error-rate widget reflects *instance-level* health-check failures (network/OS-level), not *application-level* error responses — a genuinely elevated 500-error rate with a healthy EC2 instance would show 0% "error rate" on this dashboard, which could be misleading for on-call triage.

Consider supplementing with a query against `ServerRequest` for a true application error rate.
- Static, long-lived AWS credentials are reused across S3, SES, and CloudWatch rather than scoped per-service (e.g. via separate IAM users/roles with least-privilege policies) — a compromise of one integration's credentials compromises all three.

---

6. Google OAuth 2.0 (Configured, Not Wired Up)

`src/config/passport.js` defines a full `PassportUtil` class: - `express-session`-backed session middleware (`SESSION_SECRET`, 24h cookie) - `passport.initialize()` / `passport.session()` - A `GoogleStrategy` configured with `GOOGLE_CLIENT_ID`/`GOOGLE_CLIENT_SECRET` and a **hardcoded** `callbackURL: "http://localhost:5000/auth/google/callback"` (would not work in any deployed environment as-is — no env-var override) - `serializeUser`/`deserializeUser` are no-op pass-throughs (store/retrieve the whole Google profile object in the session, not mapped to an internal Admin/TenantStaff/Client record)

However: in `src/server.js`, the instantiation is commented out —

```
// new PassportUtil(this.app)
```

— and no route anywhere in `src/features/auth/*` references `passport.authenticate("google", ...)` or mounts a `/auth/google` endpoint. This integration is **entirely dead code** in the active `src/` tree: the dependency is installed (`passport`, `passport-google-oauth20`), the strategy is fully configured, but nothing in the running application ever uses it. All real authentication goes through the custom JWT-based `adminProtect`/`staffProtect`/`clientProtect` middleware (see ARCHITECTURE.md §5.3), which has no relationship to Passport/Google OAuth at all.

Recommendation: either finish wiring this up (uncomment, fix the hardcoded callback URL, add the route, and map the Google profile to an actual user record) or remove it along with its dependencies and the `SESSION_SECRET`/`GOOGLE_CLIENT_ID`/`GOOGLE_CLIENT_SECRET` env vars to reduce unused attack surface and configuration confusion.

---

7. Socket.IO (Realtime Library, Self-Hosted)

Not a third-party *service* (no external account/API key), but included here since it's a significant third-party *library* dependency providing realtime functionality analogous to how a hosted service like Pusher/Ably would. Covered in depth in ARCHITECTURE.md §10.2. Summary: bound to the same HTTP server as the Express app (`src/config/socket.js`), CORS fully open (`origin: "*"` — notably **more permissive than the Express CORS policy** in `server.js`, which allow-lists specific origins; worth aligning these two policies), room-based addressing (`${userType}_${userId}`), used for chat messages and push notifications. Not available to the cron-worker process (no HTTP server to bind to there).

---

8. Present in `package.json` but Not Confirmed Wired Up in This Scan

Package	Likely purpose	Status
`speakeasy`	TOTP secret generation/ verification for 2FA	`Admin`/`TenantStaff` both have an `auth2FADone` boolean and `authType`/`authQuestion` fields in the schema, and `SuperAdminChoices`/ `TenantAdminChoices` have `Authenticator2FA` toggles — strongly suggesting 2FA is a real, schema-backed feature, but this scan did not trace the actual `speakeasy` call sites (likely in `src/features/auth/*`) to confirm the current implementation. Recommend a follow-up scan of the `auth` feature module specifically if 2FA behavior needs to be documented in detail.
`qrcode`	Rendering a scannable QR code for TOTP enrollment (pairs naturally with `speakeasy`)	Same caveat as above.
`bcryptjs` **and** `argon2`	Password hashing	Both a legacy (`bcryptjs`) and a modern (`argon2`) hashing library are present simultaneously — possibly indicating an in-progress migration from bcrypt to argon2 (or historical accounts still verified against bcrypt hashes while new ones use argon2). Worth confirming which models/ flows use which, to ensure old-format hashes are still verifiable and new signups use the stronger algorithm.
`pdfkit`	PDF generation	Backs `src/utilities/ invoicePdfGenerator.js` and likely `clinicalReportGenerator.js` (not Stripe/AWS-related, but worth noting as a non-trivial in-process “integration” with its own font/layout API surface) — not a network-calling third party, so not detailed further here.

Package	Likely purpose	Status
`rabbitmq` config file (`src/config/rabbitMQ.js`)	Message queue for background job processing	**Entirely commented out.** The `amqp` import and the whole `RabbitMQ` class body are block-commented; nothing instantiates or imports this file elsewhere in the codebase. Either an abandoned integration or a placeholder for future work — currently inert. The `amqplib` package does not even appear as a dependency in `package.json`, so this file would fail to run if uncommented without first adding that dependency.

9. Cross-Cutting Observations

1. **AWS credential sprawl, single IAM identity.** S3, SES, and CloudWatch all read the same `AWS_ACCESS_KEY_ID`/`AWS_SECRET_ACCESS_KEY`/`AWS_REGION` pair. There's no evidence of scoped IAM roles per service. For a healthcare-adjacent product (the schema strongly suggests ABA/behavioral-health clinical data — see DATA_MODEL.md §4.8/§4.16), tightening this to least-privilege, per-service credentials (or better, IAM instance roles instead of long-lived static keys) would meaningfully reduce blast radius from a leaked credential.
2. **Two email providers, no unified abstraction.** A shared `EmailProvider` interface with SES and SMTP as interchangeable backends (rather than two independently-called singletons with different method signatures and error-handling conventions) would make it easier to reason about deliverability, retries, and template consistency platform-wide.
3. **Dead integrations left in the tree** (Google OAuth fully configured but unmounted; RabbitMQ fully commented out) increase cognitive load for anyone auditing "what does this service talk to" and slightly inflate the dependency/attack surface (installed OAuth libraries, unused session middleware/secret) without providing value. Recommend either finishing or removing both.
4. **Webhook security is solid where implemented.** The Stripe webhook path is the standout example of doing this right: raw-body preservation, signature verification, idempotency via a dedicated ledger table, and a serializable-transaction guard against race conditions with the client-confirmation path. This pattern (idempotency ledger + signature verification + raw body) would be worth replicating if/when any other inbound webhook integration (e.g. a future SES bounce/complaint webhook, or a payer/clearinghouse EDI callback) is added.
5. **Monitoring blind spot.** The CloudWatch-powered "error rate"/"uptime" dashboard metrics are infrastructure-level proxies, not application-level truth — the actual ground truth (`ServerRequest.statusCode >= 500`) already exists in Postgres and is currently unused for this purpose (see §5.3).